

An
Industrial Training Report
On
Cyber Security
Submitted partial fulfillment for the award of
degree of
Bachelor of Technology
In
Computer Science and Engineering



Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT/CYBER SECURITY Dept.

Submitted By:

Komal Gurjar

24EGJCS108

Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Session: 2025-26

Acknowledgement

I express my sincere gratitude to **Mr. AshaRam Gurjar**, Assistant Professor, Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for providing me the opportunity to undergo Industrial Training in the domain of **Cyber Security**. His valuable guidance, continuous support, and insightful feedback played a significant role in enhancing my understanding of Network Security.

I extend my heartfelt thanks to the entire team of **GIT, Jaipur** for their support and encouragement throughout the training period. Their expertise, real-time examples, and project-based mentoring helped me gain practical exposure to modern technologies such as Parrort Security.

I am also thankful to Dr. I. C. Shrama, Principal of GIT, and Dr. Pradeep Jha, Head of the Department of Computer Science and Engineering, for their motivation and academic guidance, which inspired me to complete this industrial training successfully.

Lastly, I express my deep gratitude to my parents, friends, and fellow learners for their constant encouragement, cooperation, and moral support during the entire training duration.

Place: GIT, Jaipur

Komal Gurjar

24EGJCS108

B.Tech III Semester, II Year, CSE

Certificate



Abstract

This industrial training focused on gaining foundational and practical skills in the domain of Cyber Security, covering essential concepts such as network security, ethical hacking, cryptography, system security, and vulnerability assessment. The training followed a practical, hands-on learning approach, enabling me to understand how modern systems, networks, and applications are protected against emerging cyber threats.

Throughout the program, I learned the fundamentals of networking, Linux command-line operations, attack methodologies, and the cyber kill chain. I practiced using industry-standard tools such as **Nmap**, **Wireshark**, **Hydra**, **Burp Suite**, **Metasploit**, and **Bettercap**, which enhanced my ability to detect vulnerabilities and analyze network traffic. The training also introduced me to **OWASP Top 10** web application vulnerabilities, helping me understand real-world attack vectors like SQL Injection, XSS, CSRF, and session hijacking.

A major component of the training involved understanding and implementing cryptography. I developed a practical mini-project titled “**Secure File Encryption and Decryption System**”, where I built a tool to convert plaintext files into encrypted data using AES encryption and later restore them back to readable format. This project strengthened my understanding of encryption algorithms, key management, and secure file handling.

Overall, the training enhanced my analytical thinking, problem-solving skills, and practical knowledge of cyber defense strategies. It provided me with the confidence and technical expertise required to pursue advanced learning and professional opportunities in cyber security.

Table of Contents

• Certificate.....	ii
• Acknowledgement.....	iii
• Abstract	iv
• Table of Contents	v
• List of Figures	vi
• List of Symbols	vii

Chapters

• Chapter 1: Introduction to Cyber Security & Its Importance	01
• Chapter 2: Networking Fundamentals for Cyber Security	04
• Chapter 3: Linux Essentials & Command-Line Fundamentals	08
• Chapter 4: Ethical Hacking & Penetration Testing Basics	13
• Chapter 5: Web Application Security & OWASP Top 10	18
• Chapter 6: Cryptography, Hashing & Secure File Handling	24
• Chapter 7: Cyber Security Tools, Vulnerability Analysis & Practical Labs..	29
• My Project: Secure File Encryption & Decryption System	34
• Training Summary	41
• Key Outcomes	42
• References	43

List of figures

• Figure 01: Cyber Security Architecture Overview	03
• Figure 02: OSI Model Layered Structure	06
• Figure 03: TCP/IP Model Diagram	06
• Figure 04: Packet Flow in a Network (Source to Destination)	07
• Figure 05: Linux File System Hierarchy	16
• Figure 06: Cyber Attack Kill Chain Stages	16
• Figure 07: OWASP Top 10 Vulnerabilities Overview Diagram	19
• Figure 08: Symmetric vs Asymmetric Encryption Workflow	26
• Figure 09: File Encryption & Decryption Process Flowchart	30
• Figure 10: Wireshark Packet Analysis Interface.....	31
• Figure 11: Burp Suite Interception Proxy Window.....	32
• Figure 11: Burp Suite Interception Proxy Window.....	33
• Snapshot 1: Nmap Scan Output	38
• Snapshot 2: Burp Suite Intercept Mode	38
• Snapshot 3: File Encryption Tool Output	39

List Of Tables

Table 01: Comparison of OSI Model vs TCP/IP Model	02
Table 02: Common Networking Ports and Their Services	09
Table 03: Types of Cyber Attacks and Their Descriptions	16
Table 04: Linux Commands and Their Uses in Cyber Security	22
Table 05: OWASP Top 10 Vulnerabilities Summary	26
Table 06: Symmetric vs Asymmetric Encryption Comparison	30
Table 07: Cyber Security Tools and Their Functional Areas	33

List OF Symbols

CIA — Confidentiality, Integrity, Availability
OSI — Open Systems Interconnection Model
TCP/IP — Transmission Control Protocol / Internet Protocol
IP — Internet Protocol
MAC — Media Access Control
DoS — Denial of Service
DDoS — Distributed Denial of Service
SQLi — SQL Injection
XSS — Cross-Site Scripting
CSRF — Cross-Site Request Forgery
IDS — Intrusion Detection System
IPS — Intrusion Prevention System
VPN — Virtual Private Network
AES — Advanced Encryption Standard
RSA — Rivest–Shamir–Adleman Encryption Algorithm
HASH — One-Way Cryptographic Hash
MITM — Man in the Middle Attack
VAPT — Vulnerability Assessment & Penetration Testing
HTTP/HTTPS — Hypertext Transfer Protocol / Secure HTTP
Nmap — Network Mapper
SIEM — Security Information and Event Management

Chapter 1

Introduction to Cyber Security & Its Importance

Today marked the beginning of my industrial training in Cyber Security, one of the most crucial domains in the digital era. The session introduced me to the core principles of cyber security, the growing need for secure systems, and how organizations defend themselves from digital threats. I learned how cyber security focuses on protecting computer systems, networks, applications, and data from unauthorized access, attacks, and damage. With cyber threats increasing globally, the importance of skilled professionals in this field has become essential for every industry, from banking and healthcare to government and IT.

Overview of Cyber Security

Cyber Security refers to the techniques, technologies, and processes used to safeguard digital assets. The instructor explained that cyber security works on protecting data and systems while ensuring smooth business operations without interruptions caused by cyber attacks.

Cyber Security mainly focuses on three foundational principles called the **CIA Triad**:

- **Confidentiality**: Ensuring only authorized users access data
- **Integrity**: Ensuring data is accurate and unaltered
- **Availability**: Ensuring systems and data are accessible when required

Why Cyber Security Is Important?

Cyber Security has become critical due to the increasing number of online services and digital transactions. The instructor highlighted several reasons why cyber security has become a top priority:

- Rising cyber crimes such as ransomware and phishing
- Sensitive user information stored online
- Growth of cloud computing
- Increasing dependency on smartphones and internet services

Types of Cyber Attacks

. Malware Attacks

Viruses, worms, trojans, ransomware, and spyware that harm systems or steal data.

2. Phishing & Social Engineering

Tricking users into giving sensitive information such as passwords or OTPs.

3. SQL Injection (SQLi)

Injecting malicious queries to access or modify database information.

4. Denial of Service (DoS) / DDoS

Overloading a server with traffic until it stops responding.

5. Man-in-the-Middle (MITM)

Intercepting communication between two parties.

6. Password Attacks / Brute Force

Cracking weak passwords using automated tools.

Cyber Attack Lifecycle

A cyber Attack usually follows a structured process known as the **Cyber Kill Chain**:

1. **Reconnaissance** – Gathering information
2. **Scanning** – Identifying vulnerable ports/services
3. **Gaining Access** – Exploiting vulnerabilities
4. **Maintaining Access** – Installing backdoors
5. **Covering Tracks** – Removing logs/traces

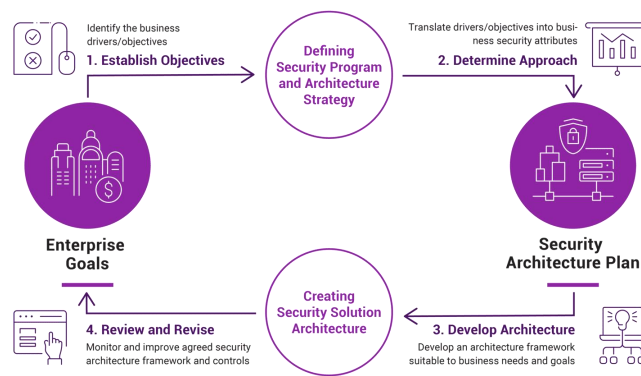


figure 01: Cyber Security Architecture Overview

Skills Required for Cyber Security

- Networking Fundamentals
- Linux Operating System
- Cyber Attacks & Tools
- Ethical Hacking Techniques
- Vulnerability Assessment
- Incident Detection
- Cryptography
- Web Application Security

Development Environment Setup for Cyber Security

- kali Linux / Parrot OS
- Nmap
- Wireshark
- Burp Suite
- Metasploit
- Hydra
- Bettercap

Chapter 2

Networking Fundamentals for Cyber Security

networking is the backbone of cyber security because almost every cyber attack—whether it is malware, phishing, or web exploitation—travels through a network.

The instructor emphasized that without strong networking knowledge, it is impossible to become a cyber security expert or ethical hacker. Concepts like IP addresses, ports, OSI layers, protocols, routing, and packet flow help in analyzing attacks, scanning systems, detecting vulnerabilities, and monitoring traffic.

Introduction to Networking Basics

Networking is the process of connecting multiple devices so they can communicate and share data. In cyber security, understanding how communication happens helps identify suspicious patterns, detect intrusions, and secure critical systems.

Every communication between two devices follows these steps:

1. Sender creates data
2. Data is broken into packets
3. Packets travel through routers/switches
4. Receiver reconstructs packets

IP Address, MAC Address & Ports

1. IP Address (Logical Address)

Used to identify devices on a network.

Example: 192.168.1.10

2. MAC Address (Physical Address)

A unique hardware address assigned to a device's network interface.

Example: 8C-85-90-1D-7A-BC

3. Ports

Ports identify specific services running on a computer.

Examples:

- Port 80 – HTTP
- Port 443 – HTTPS
- Port 22 – SSH
- Port 21 – FTP
- Port 445 – SMB (File sharing)

Hackers use port scanning tools like Nmap to find open ports that can be exploited.

OSI Model (Open Systems Interconnection)

The OSI model explains how data travels from one device to another. It has seven layers:

1. Physical – Cables, signals
2. Data Link – MAC address, frames
3. Network – IP address, routing
4. Transport – TCP/UDP
5. Session – Communication control
6. Presentation – Encryption/Decryption
7. Application – Browser, email, apps



Figure 02: OSI Model Layered Structure

TCP/IP Model

Compared to OSI, the TCP/IP model is simpler and used practically in real-world networking.

Layers include:

1. Application Layer
2. Transport Layer
3. Internet Layer
4. Network Access Layer

how TCP/IP is the foundation of internet protocols, and understanding it helps in analyzing attacks like MITM, packet sniffing, and DoS.

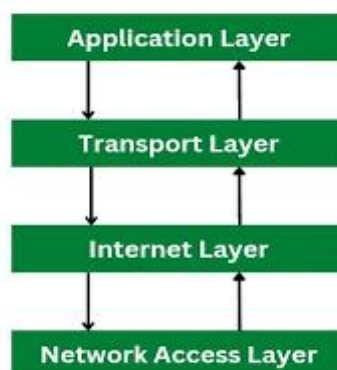


Figure 03: TCP/IP Model Diagram

learning how packets move through routers and switches is essential for penetration testing.

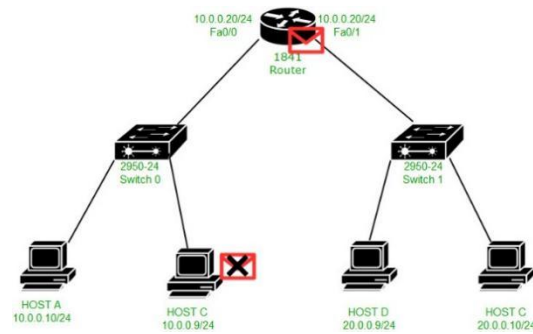


Figure 04: Packet Flow in a Network

common Networking Tools Used in Cyber Security

Tool	Purpose
Nmap	Port scanning
Wireshark	Packet capturing
Traceroute	Network path detection
Netcat (nc)	Communication & reverse shells
IPTables	Firewall management

Table 02: Common Networking Ports and Their Services

Importance of Networking in Cyber Security

The instructor concluded that networking is the foundation of ethical hacking, because:

- Every attack starts with reconnaissance on a network
- Traffic analysis reveals suspicious behavior
- Ports & protocols reveal entry points
- Logs help identify intrusions

Chapter 3

Linux Essentials & Command-Line Fundamentals

Linux one of the most essential operating systems for cyber security professionals. Linux most real-world servers, security tools, penetration testing tools, and hacking frameworks run on Linux-based systems such as Kali Linux, Parrot OS, and Ubuntu. Understanding Linux commands and file structure is necessary for performing ethical hacking, analyzing logs, managing network services, and using cyber security toolkits.

Linux provides flexibility, stability, open-source tools, and a powerful command-line environment—making it the preferred OS for cyber security.

Introduction to Linux

Linux is an open-source operating system widely used in:

- Ethical Hacking
- Cloud Servers
- IoT Devices
- DevOps
- Network Security
- Web Servers
- Security Operations Centers

Why Linux is Important in Cyber Security

Some key reasons include:

- Most hacking tools are available only on Linux
- Strong terminal-based control
- Better file permissions & security
- High stability for servers

Linux File System Structure

Linux follows a hierarchical directory structure. The instructor explained the main directories:

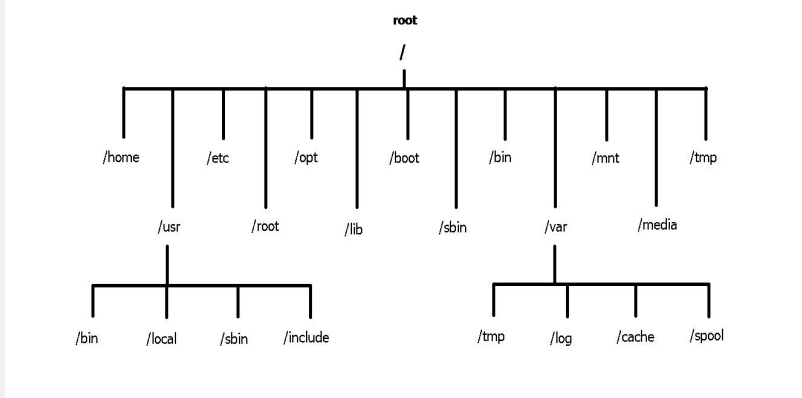
/home	-User files
• /etc	-Configuration files
• /var	-Log files, variable data
• /usr	-User programs & utilities
• /bin	-Basic commands
• /sbin	-System commands
• /tmp	-Temporary files
• /root	-Administrator home directory
 <pre>graph TD root["root /"] --- home["/home"] root --- etc["/etc"] root --- opt["/opt"] root --- boot["/boot"] root --- bin["/bin"] root --- mnt["/mnt"] root --- tmp["/tmp"] root --- usr["/usr"] root --- root_dir["/root"] root --- lib["/lib"] root --- sbin["/sbin"] root --- var["/var"] root --- media["/media"] usr --- usr_bin["/bin"] usr --- usr_local["/local"] usr --- usr_sbin["/sbin"] usr --- usr_include["/include"] var --- var_tmp["/tmp"] var --- var_log["/log"] var --- var_cache["/cache"] var --- var_spool["/spool"]</pre> File System Hierarchy(FHS) of Linux	

figure 05: Linux File System Hierarchy

Basic Linux Commands

1. Navigation Commands

- **ls** — List files
- **cd** — Change directory
- **pwd** — Print working directory

2. File Commands

- `cat file.txt` — View file
- `touch file.txt` — Create file
- `cp file1 file2` — Copy
- `mv file1 file2` — Move/Rename
- `rm file.txt` — Delete file

3. Directory Commands

- `mkdir folder` — Create directory
- `rmdir folder` — Remove directory
- `tree` — Directory tree view

4. Permission Commands

- `chmod 755 file` — Change permissions
- `chown user file` — Change ownership

Commands learned:

- `ps aux` — List processes
- `top` — Real-time process monitoring
- `systemctl start service` — Start service
- `systemctl stop service` — Stop service
- `kill PID` — Terminate process

Networking Commands in Linux

These commands are widely used in penetration testing:

- `ifconfig / ip a` — Show network interfaces
- `ping` — Check connectivity
- `netstat -tulnp` — View open ports
- `nmap` — Perform scanning
- `arp -a` — View ARP table
- `route -n` — View routing table

Package Management

Linux uses package managers to install tools.

- `apt update`
- `apt install toolname`
- `apt remove toolname`



18 MOST USED LINUX COMMANDS		
ls Lists files and directories in the current directory.	pwd Shows the current working directory.	ssh Connects to a remote machine using SSH protocol.
cd Changes the current directory.	cat Concatenates and displays file content.	sudo Executes a command with root/admin privileges.
pwd Prints the current working directory.	grep Searches for a specified pattern in files.	top Displays system resource usage and running processes.
mkdir Creates a new directory.	find Searches for files and directories.	wget Downloads files from the web.
rm Removes files and directories.	chmod Changes the permissions of files and directories.	tar Archives files and directories into a single file.
cp Copies files and directories.	chown Changes the ownership of files.	zip Compresses files using the zip algorithm.

Table 04: Linux Commands and Their Uses in Cyber Security

Importance of Linux in Ethical Hacking

Linux is heavily used in hacking because:

- It offers full system control
- It supports powerful networking tools
- Logs are accessible and modifiable
- Network configurations are easy to audit

Chapter 4

Ethical Hacking & Penetration Testing Basics

Ethical Hacking one of the most important and practical components of Cyber Security. Ethical hacking involves legally identifying, testing, and fixing vulnerabilities in computer systems, networks, and applications. The trainer explained that ethical hackers use the same tools and techniques as attackers, but with permission, to strengthen the security of an organization.

what is Ethical Hacking?

Ethical hacking is the authorized practice of testing systems for vulnerabilities. Unlike malicious hackers, ethical hackers work with permission to improve security.

Types of Hackers

- White Hat Hackers – Ethical hackers
- Black Hat Hackers – Malicious attackers
- Grey Hat Hackers – Mix of both intentions

Need for Ethical Hacking

- Companies store large amounts of sensitive data
- Systems are vulnerable to attacks
- Web applications often have security flaws
- Employees may fall for phishing attacks
- Organizations face continuous cyber threats
- Network misconfigurations are common

Penetration Testing (Pentesting)

Penetration Testing is a simulated cyber attack performed on a system to find vulnerabilities. It involves analyzing network weaknesses, misconfigurations, insecure applications, weak passwords, and improper security settings.

Types of Penetration Testing

1. Network Pentesting
 - Identifies open ports, insecure services, weak configurations
2. Web Application Pentesting
 - Detects flaws like SQLi, XSS, CSRF, Authentication bypass
3. Wireless Pentesting
 - Analyzes Wi-Fi security and encryption
4. Mobile Application Pentesting
 - Tests Android/iOS apps
5. Cloud Pentesting
 - Evaluates cloud-based environments
6. Social Engineering Pentesting
 - Tests human vulnerabilities

Phases of Ethical Hacking

Ethical hacking follows a structured methodology known as the 5-Phase Hacking Cycle.

1. Reconnaissance (Information Gathering)

Collecting data about the target using:

- Google dorking
- Whois lookup
- Social media
- Email harvesting

2. Scanning (Finding Weaknesses)

Using tools to detect:

- Open ports
- Running services
- Vulnerabilities
- Hosts & networks

Tools used: Nmap, Nessus, Nikto

3. Gaining Access (Exploitation)

Exploiting vulnerabilities to enter a system using:

- SQL Injection
- Weak passwords
- Misconfigurations
- Default credentials

4. Maintaining Access

Attackers may install:

- Backdoors
- Reverse shells
- Malware persistence

5. Covering Tracks

Clearing logs to avoid detection.

Ethical hackers document all steps instead of hiding activities.



Figure 06: Cyber Attack Kill Chain Stages

Common Tools Used in Penetration Testing

Tool	Purpose
Nmap	Scanning ports/services
Burp Suite	Web app testing
Metasploit	Exploitation
Hydra	Password attacks
Nikto	Web server scanning
Wireshark	Traffic analysis
Aircrack-ng	Wi-Fi pentesting
John the Ripper	Password cracking
Bettercap	MITM attacks

Pentesting Approaches

1. Black Box Testing

Tester has no information about the target.

2. White Box Testing

Tester has full access to system details.

3. Gray Box Testing

web Application Pentesting Basics

- njection
- Broken Authentication
- Sensitive Data Exposure
- Insecure Direct Object Reference
- XSS
- Security Misconfiguration
- Cross-Site Request Forgery
- Using Components with Known Vulnerabilities
- Insufficient Logging & Monitoring

These topics will be covered in detail in the next chapter.

Legal & Ethical Considerations

Ethical hacking must always follow legal guidelines:

- Written permission is mandatory
- Testing must be within scope
- No data should be harmed

Chapter 5

Web Application Security & OWASP Top 10

web Application Security an essential domain in cyber security. Since most modern applications run on web platforms, securing them from cyber attacks has become a top priority. The instructor introduced the OWASP Top 10, which is a widely accepted standard that identifies the most critical security risks in web applications.

I learned how attackers exploit weaknesses in web servers, databases, authentication systems, sessions, and user inputs. Understanding these vulnerabilities helps ethical hackers secure applications, protect user data, and prevent system compromise.

Introduction to Web Application Security

Web application security is the practice of protecting websites and online services from cyber threats that exploit vulnerabilities in:

- Login and authentication systems
- Databases
- Input fields (forms, search bars)
- Cookies and sessions
- API endpoints
- Server configurations

A small vulnerability can lead to:

- Data breaches
- Account takeovers
- Unauthorized access
- Database leaks
- Website defacement

HTTP vs HTTPS

HTTP (Hypertext Transfer Protocol)

- Sends data in plain text
- Vulnerable to MITM attacks

HTTPS (Secure HTTP)

- Uses SSL/TLS encryption
- Secures login pages, payment portals, and forms

Using HTTPS is the first step toward securing a web application.

What is OWASP?

OWASP (Open Web Application Security Project) is a non-profit organization dedicated to improving software security. It publishes the OWASP Top 10, which lists the most dangerous web application vulnerabilities.

The OWASP Top 10 acts as a guide for penetration testers, security analysts, and developers to understand and fix critical web security flaws.



Figure 07: OWASP Top 10 Vulnerabilities Overview Diagram

OWASP Top 10 Vulnerabilities (Explained)

Below are the most critical vulnerabilities every ethical hacker must understand:

A1: Broken Access Control

Occurs when users can access information or functions they shouldn't.

Example attacks:

- IDOR (Insecure Direct Object Reference)
- Privilege escalation

A2: Cryptographic Failures (Sensitive Data Exposure)

Weak or missing encryption can expose sensitive data like:

- Passwords
- Banking information
- Personal details

Attackers use sniffing or database leaks to steal this information.

A3: Injection Attacks (SQL Injection, Command Injection)

Attackers insert malicious code in input fields.

Example: ' OR '1'='1

Effects:

- Database dumping
- Data deletion
- Admin login bypass

Tools used: SQLMap, Burp Suite

A4: Insecure Design

Poorly designed systems are vulnerable even before coding begins.

Example:

- Password reset without OTP
- No rate limiting

A5: Security Misconfiguration

One of the most common vulnerabilities.

Examples:

- Default passwords
- Open cloud storage
- Misconfigured firewalls

A6: Vulnerable and Outdated Components

Using outdated libraries, plugins, and frameworks exposes apps to known exploits.

Example: Running old versions of PHP, Apache, or WordPress

A7: Identification & Authentication Failures

Weak authentication allows attackers to take over accounts.

Examples:

- No lockout after multiple wrong passwords
- Weak OTP verification
- Session hijacking

A8: Software & Data Integrity Failures

Occurs when applications rely on untrusted or tampered data.

Example:

- Malicious updates
- Unverified JSON or XML input

A9: Security Logging & Monitoring Failures

Lack of monitoring results in:

- Late detection of attacks
- Missing evidence during investigations

Example:

- No logs for login attempts

A10: Server-Side Request Forgery (SSRF)

Allows attackers to make the server send requests to internal systems.

Example:

A website's image upload feature can be exploited to access internal IP addresses.

Common Web Application Attack Techniques

1. Cross-Site Scripting (XSS)

Injecting malicious scripts into websites.

Example payload:

```
<script>alert('Hacked')</script>
```

2. Cross-Site Request Forgery (CSRF)

Forces users to perform actions without their consent.

3. Clickjacking

Tricking users into clicking hidden buttons.

4. Brute Force Attacks

Guessing passwords using automation tools.

Tool example: Hydra

5. Directory Traversal

Accessing restricted server files.

Example:

- ../../etc/passwd

Table 05: OWASP Top 10 Vulnerabilities Summary

--

Importance of Web Application Security

Web application security protects:

- User data
- Website functionality
- Business reputations
- Financial transactions
- API endpoints

Chapter 6

Cryptography, Hashing & Secure File Handling

cryptography, one of the most important pillars of cyber security. Cryptography ensures that sensitive information remains protected from unauthorized access by converting readable data into unreadable form. This chapter explained the working of encryption, decryption, hashing, digital signatures, and secure file handling methods used in real-world applications.

Cryptography plays a critical role in web applications, secure communication, online transactions, password storage, digital documents, backups, and network communications.

Introduction to Cryptography

Cryptography is the technique of securing information by transforming it into a form that cannot be understood by unauthorized users.

The key goals of cryptography (CIA+) include:

- **Confidentiality** – keeping data secret
- **Integrity** – keeping data accurate
- **Authentication** – verifying user identity
- **Non-Repudiation** – preventing denial of actions

Types of Cryptographic Techniques

Cryptography is mainly divided into two categories:

1. Symmetric Key Cryptography

Uses **one single key** for both encryption and decryption.

Examples of Symmetric Algorithms

- AES (Advanced Encryption Standard)
- DES
- 3DES
- Blowfish
- RC4

Usage

- File encryption
- Database encryption
- Wi-Fi security (WPA2 uses AES)

2. Asymmetric Key Cryptography

Uses **two different keys**:

- **Public Key** – Used for encryption
- **Private Key** – Used for decryption

Examples

- RSA
- ECC
- Diffie-Hellman

Encryption & Decryption Workflow

The instructor showed how encryption works in real-world systems:

1. **Plain Text** → readable message
2. **Encryption Algorithm** → converts it
3. **Cipher Text** → unreadable data
4. **Decryption Algorithm**
5. **Back to Plain Text**

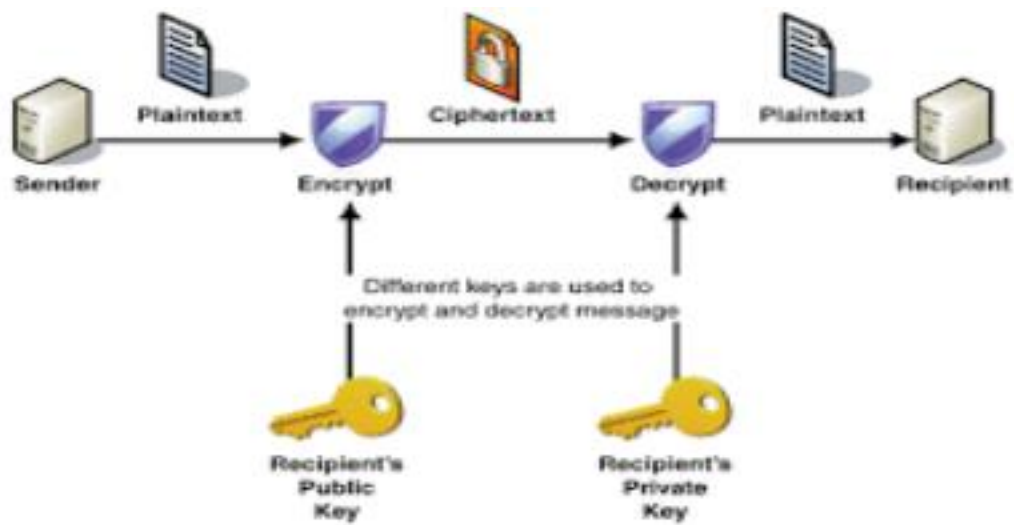


Figure 08: Encryption–Decryption Workflow Diagram

Hashing in Cyber Security

Hashing converts data into a fixed-size value using a one-way function. Hashes **cannot be reversed**, unlike encryption.

Common Hashing Algorithms

- MD5
- SHA-1
- SHA-256 (most secure widely used)
- bcrypt
- PBKDF2
- Argon2

Hashes are used for:

- Storing passwords
- File integrity checking
- Digital forensics
- Detecting malware

Example hash output:

Input: hello

SHA-256 Hash: 2cf24dba5f...

Hash Cracking

Attackers use tools to crack weak hashes:

- **John the Ripper**
- **Hashcat**
- **Hydra**

Hashes can be cracked if:

- Weak password is used
- MD5/SHA1 is used
- Rainbow tables exist

Secure hashing uses **salt + slow hashing algorithms** like bcrypt or Argon2.

File Encryption Concepts (Useful for Your Project)

steps for file encryption:

1. Select encryption algorithm (AES)
2. Generate secret key
3. Read file → convert to bytes
4. Encrypt file → output cipher data
5. Save encrypted file
6. Use same key to decrypt file back to original

Tools Used in Cryptography

Tool	Purpose																																																																																										
OpenSSL	Encryption/Decryption, generating keys																																																																																										
GPG	File encryption																																																																																										
VeraCrypt	Disk encryption																																																																																										
Hashcat	Hash cracking																																																																																										
John the Ripper	Password cracking																																																																																										
Base64	Encoding/decoding																																																																																										
	<table><tr><th>Algorithm</th><th>Created By</th><th>Year</th><th>Key Size</th><th>Block Size</th><th>Rounds</th><th>Structure</th><th>Flexible</th><th>Features</th></tr><tr><td>DES</td><td>IBM</td><td>1975</td><td>64 bits</td><td>64bits</td><td>16</td><td>Festial</td><td>No</td><td>Not Strong Enough</td></tr><tr><td>DH</td><td>Whitfield Diffie and Martin Hellman</td><td>1976</td><td>Variable</td><td>-</td><td>-</td><td>Public key Algorithm</td><td>Yes</td><td>Good Security and Low Speed</td></tr><tr><td>E-DES</td><td>IBM</td><td>1977</td><td>1024 bits</td><td>128 bits</td><td>16</td><td>Festial</td><td>-</td><td>Good Security and fast Speed</td></tr><tr><td>RSA</td><td>Rivest Shamir Adleman</td><td>1977</td><td>1024 to 4096</td><td>128 bits</td><td>1</td><td>Public Key Algorithm</td><td>No</td><td>Excellent Security and Low Speed</td></tr><tr><td>T-DES</td><td>IBM</td><td>1978</td><td>112 or 168</td><td>64 bits</td><td>48</td><td>Festial</td><td>Yes</td><td>Adequate Security and fast</td></tr><tr><td>ECC</td><td>Neal Koblitz and Victor Miller</td><td>1985</td><td>More than symmetric and variable</td><td>Variable</td><td>1</td><td>Public Key Algorithm</td><td>Yes</td><td>Excellent Security and fast Speed</td></tr><tr><td>EEE</td><td>Taher Elgamal</td><td>1985</td><td>1024 bits</td><td>-</td><td>-</td><td>Public Key Algorithm</td><td>Yes</td><td>Enough secured and fast Speed</td></tr><tr><td>RC4</td><td>Ron Rivest</td><td>1987</td><td>Variable</td><td>40-2048</td><td>256</td><td>Festiel Stream</td><td>Yes</td><td>fast Cipher</td></tr><tr><td>RC2</td><td>Ron Rivest</td><td>1987</td><td>8,128,64 by</td><td>64 bits</td><td>16</td><td>Festiel</td><td>-</td><td>Good and fast Security</td></tr></table>	Algorithm	Created By	Year	Key Size	Block Size	Rounds	Structure	Flexible	Features	DES	IBM	1975	64 bits	64bits	16	Festial	No	Not Strong Enough	DH	Whitfield Diffie and Martin Hellman	1976	Variable	-	-	Public key Algorithm	Yes	Good Security and Low Speed	E-DES	IBM	1977	1024 bits	128 bits	16	Festial	-	Good Security and fast Speed	RSA	Rivest Shamir Adleman	1977	1024 to 4096	128 bits	1	Public Key Algorithm	No	Excellent Security and Low Speed	T-DES	IBM	1978	112 or 168	64 bits	48	Festial	Yes	Adequate Security and fast	ECC	Neal Koblitz and Victor Miller	1985	More than symmetric and variable	Variable	1	Public Key Algorithm	Yes	Excellent Security and fast Speed	EEE	Taher Elgamal	1985	1024 bits	-	-	Public Key Algorithm	Yes	Enough secured and fast Speed	RC4	Ron Rivest	1987	Variable	40-2048	256	Festiel Stream	Yes	fast Cipher	RC2	Ron Rivest	1987	8,128,64 by	64 bits	16	Festiel	-	Good and fast Security
Algorithm	Created By	Year	Key Size	Block Size	Rounds	Structure	Flexible	Features																																																																																			
DES	IBM	1975	64 bits	64bits	16	Festial	No	Not Strong Enough																																																																																			
DH	Whitfield Diffie and Martin Hellman	1976	Variable	-	-	Public key Algorithm	Yes	Good Security and Low Speed																																																																																			
E-DES	IBM	1977	1024 bits	128 bits	16	Festial	-	Good Security and fast Speed																																																																																			
RSA	Rivest Shamir Adleman	1977	1024 to 4096	128 bits	1	Public Key Algorithm	No	Excellent Security and Low Speed																																																																																			
T-DES	IBM	1978	112 or 168	64 bits	48	Festial	Yes	Adequate Security and fast																																																																																			
ECC	Neal Koblitz and Victor Miller	1985	More than symmetric and variable	Variable	1	Public Key Algorithm	Yes	Excellent Security and fast Speed																																																																																			
EEE	Taher Elgamal	1985	1024 bits	-	-	Public Key Algorithm	Yes	Enough secured and fast Speed																																																																																			
RC4	Ron Rivest	1987	Variable	40-2048	256	Festiel Stream	Yes	fast Cipher																																																																																			
RC2	Ron Rivest	1987	8,128,64 by	64 bits	16	Festiel	-	Good and fast Security																																																																																			

Table 06: Comparison of Cryptographic Techniques

Real-World Uses of Cryptography

- WhatsApp end-to-end encryption
- Online banking
- Aadhaar & PAN data protection
- Secure website logins (HTTPS)
- Cloud data storage

Practical Tools in Cyber Security

7.1 Nmap – Network Scanning Tool

Nmap (Network Mapper) is one of the most popular tools used for network discovery and security auditing. Ethical hackers use Nmap to identify open ports, services, operating systems, and potential vulnerabilities in a system.

Features of Nmap

- Port scanning
- Service enumeration
- OS detection
- Host discovery
- Version detection
- Vulnerability detection using scripts (NSE)

Basic Nmap Commands

1. Scan a single target

```
nmap 192.168.1.10
```

2. Scan multiple ports

```
nmap -p 80,443,21 192.168.1.10
```

3. Scan all ports

```
nmap -p- 192.168.1.10
```

4. OS Detection

```
nmap -O 192.168.1.10
```

5. Service Version Detection

```
nmap -sV 192.168.1.10
```

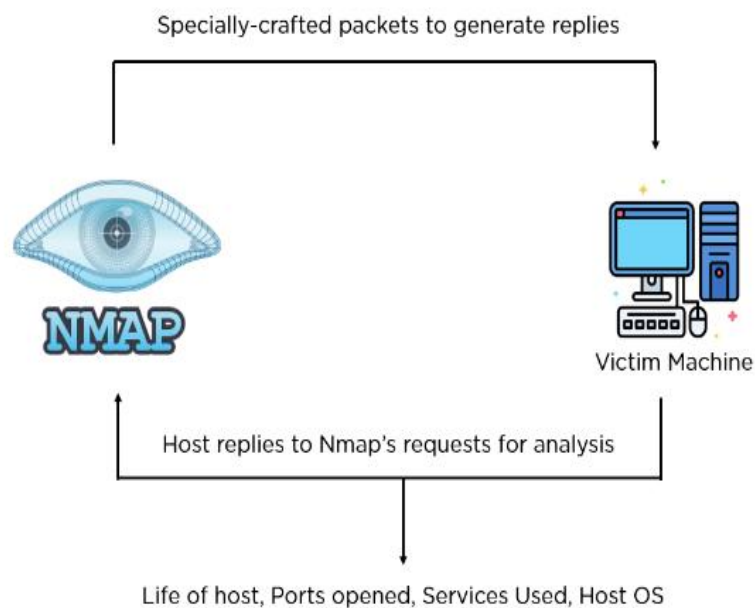


Figure 09: Nmap Network Scanning Workflow

7.2 Wireshark – Packet Sniffing & Traffic Analysis

Wireshark is a powerful tool used to capture and analyze network packets.

- Detecting anomalies
- Monitoring network traffic
- Analyzing protocols
- Identifying suspicious connections

Features of Wireshark

- Live packet capturing
- Deep packet inspection
- Filter-based analysis
- Protocol decoding
- Detecting MITM attacks

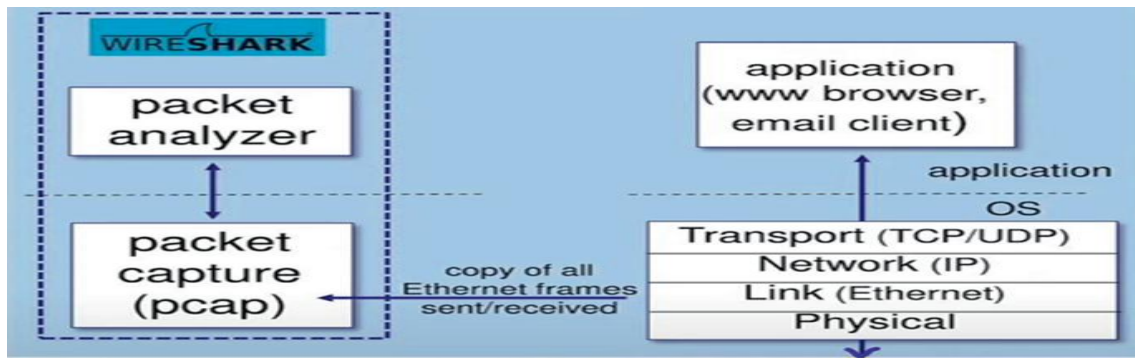


figure 10: Wireshark Packet Analysis Interface

7.3 Burp Suite – Web Application Penetration Testing

Burp Suite is a professional tool used for testing the security of web applications. It acts as an intercepting proxy that allows ethical hackers to analyze and manipulate HTTP requests and responses.

Key Modules of Burp Suite

- **Proxy** – Intercept and edit requests
- **Intruder** – Automate attacks (Bruteforce, fuzzing)
- **Repeater** – Manually edit and resend requests
- **Scanner (Pro)** – Automated vulnerability scanning
- **Decoder** – Encode/decode data
- **Comparer** – Compare two responses
- **Extender** – Install plugins

Common Uses of Burp Suite

- SQL Injection testing
- XSS testing
- CSRF testing
- Cookie & session analysis
- Login bruteforce testing

7.5 Metasploit Framework – Exploitation Toolkit

Metasploit is one of the most advanced penetration testing frameworks used for:

- Exploit development
- Payload generation
- Vulnerability exploitation
- Post-exploitation
- Backdoor creation

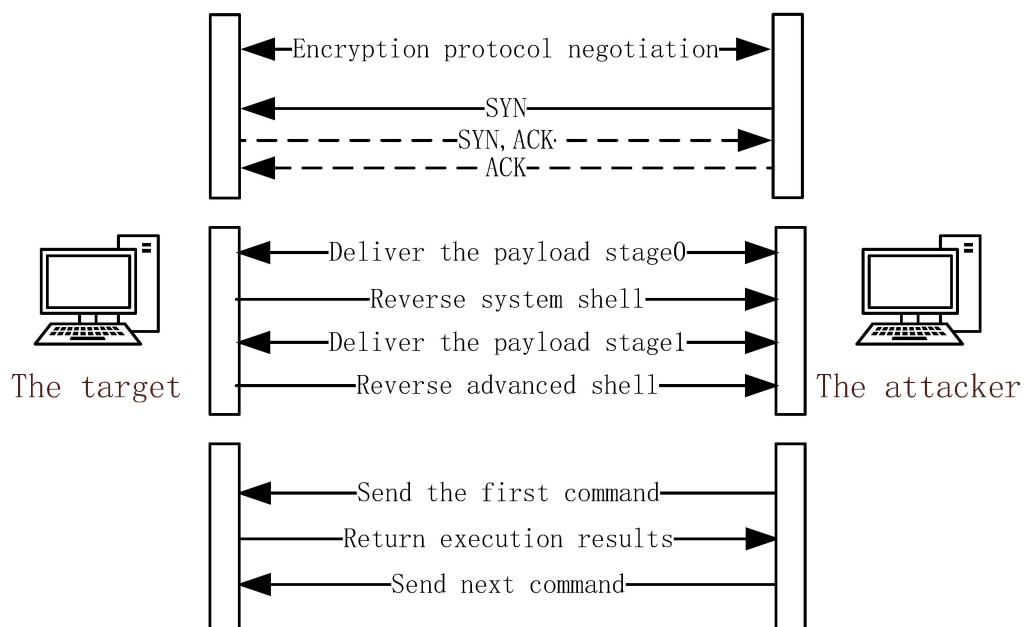


figure 12: Metasploit Exploit Execution Flow

Chapter 8

Encrypted–Decrypted File Project (Cryptography-Based Application)

As a part of my cyber security industrial training, I developed a practical project based on **file encryption and decryption** using the AES (Advanced Encryption Standard) algorithm. This project demonstrates how sensitive files can be securely protected and later restored using cryptographic techniques.

The goal of the project was to understand the **real-world implementation** of cryptography, secure file handling, key generation, and data confidentiality. This chapter explains the complete working, code, output, and implementation details of my project.

8.1 Project Overview

The main objective of the project is to:

- Encrypt a file so it becomes unreadable
- Decrypt the same file using the correct key
- Understand symmetric key cryptography
- Demonstrate real-world use of AES encryption
- Protect data from unauthorized access

This project shows how encryption works practically in cyber security, especially in:

- Secure file storage
- Confidential reports
- Personal data protection
- Digital vaults
- Backup security

8.2 Technology & Tools Used

Component	Details
Programming Language	Python
Algorithm	AES (Advanced Encryption Standard)
Library Used	cryptography (Fernet)
File Type Supported	Any file (txt, pdf, jpg, mp4, etc.)
OS Used	Windows/Linux

8.3 Workflow of the Project

Below is the entire flow of file encryption & decryption:

1. User selects a file
2. System generates an AES key
3. File is read in bytes
4. Data is encrypted
5. Encrypted file is saved
6. Same key is used for decryption
7. Original file is restored

8.4 Algorithm Used (AES Symmetric Encryption)

Encryption

1. Generate key
2. Load file
3. Convert to bytes
4. Encrypt using AES algorithm
5. Save encrypted output

Decryption

1. Load encryption key
2. Read encrypted file
3. Apply AES decryption
4. Restore original content
5. Save decrypted file

This proves that encryption is reversible **only with the correct key**.

8.5 Project Code (Complete)

encryption.py

```
from cryptography.fernet import Fernet

def generate_key():

    key = Fernet.generate_key()

    with open("secret.key", "wb") as key_file:

        key_file.write(key)

def load_key():

    return open("secret.key", "rb").read()
```

```
# Encrypt file

def encrypt_file(filename):

    key = load_key()

    fernet = Fernet(key)

    with open(filename, "rb") as file:

        data = file.read()

    encrypted_data = fernet.encrypt(data)

    with open("encrypted_" + filename, "wb") as file:

        file.write(encrypted_data)

    print("File encrypted successfully!")

generate_key()

encrypt_file("sample.txt")
```

decryption.py

```
from cryptography.fernet import Fernet

def load_key():

    return open("secret.key", "rb").read()

def decrypt_file(encrypted_filename):

    key = load_key()

    fernet = Fernet(key)

    with open(encrypted_filename, "rb") as file:

        encrypted_data = file.read()

    decrypted_data = fernet.decrypt(encrypted_data)
```

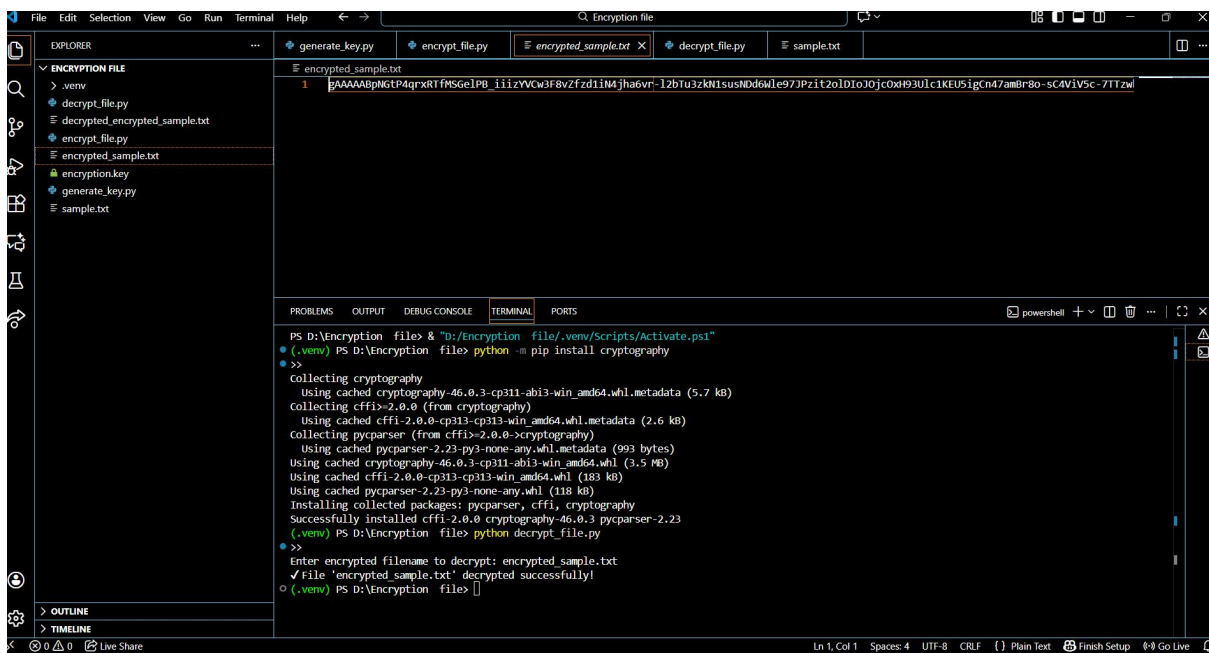
with open("decrypted_" + encrypted_filename, "wb") as file:

file.write(decrypted_data)

print("File decrypted successfully!")

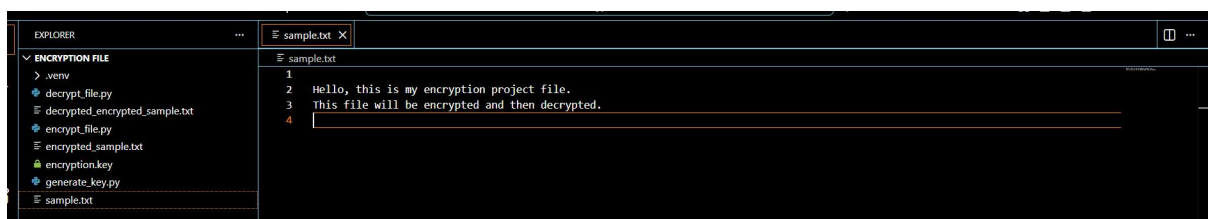
decrypt_file("encrypted_sample.txt")

8.6 Output Screenshots (Description)

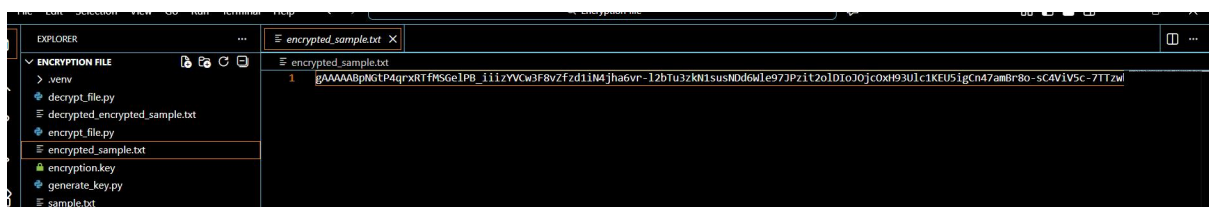


```
PS D:\Encryption file> "D:\Encryption file\venv\scripts\Activate.ps1"
(.venv) PS D:\Encryption file> python -m pip install cryptography
Collecting cryptography
  Using cached cryptography-46.0.3-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.0.0-cp313-cp313-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-2.23-py3-none-any.whl.metadata (993 bytes)
Using cached cryptography-46.0.3-cp311-abi3-win_amd64.whl (3.5 MB)
Using cached cffi-2.0.0-cp313-cp313-win_amd64.whl (183 kB)
Using cached pycparser-2.23-py3-none-any.whl (118 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed pycparser-2.23 cryptography-46.0.3
(.venv) PS D:\Encryption file> python encrypt_file.py
Enter encrypted filename to encrypt: encrypted_sample.txt
File 'encrypted_sample.txt' encrypted successfully!
(.venv) PS D:\Encryption file>
```

Snapshot 01: Original File (sample.txt)



```
1
2 Hello, this is my encryption project file.
3 This file will be encrypted and then decrypted.
4
```



```
1 gAAAAABpNGT4qrxRTfMSGeLPB_iiizYVCw3F8vZfzd1iMjha6vr-12btu3zkn1susNdd6wle97JPzit2o1DioJ0jcoxH93Ulc1KEU5igCn47amBr8o-5C4ViV5c-7TTzw
```

Snapshot 02: Encrypted File Output



Snapshot 03: Decrypted File Output

8.7 Security Analysis of the Project

Feature	Explanation
AES-level encryption	Strong and widely used
Key stored in secret.key	Must be protected
Symmetric key algorithm	Fast and efficient
Files cannot be decrypted without key	Ensures confidentiality
Works for all file types	Universal usage

8.8 Applications of the Project

This project can be used in:

- Secure file storage
- Banking & financial documents
- Personal data protection
- Industrial confidential reports
- Cyber forensics
- Company internal documents
- Secure backup systems

8.9 Challenges Faced

- Managing encryption key securely
- Large files take longer to encrypt
- Encrypted files must be stored safely
- Key loss → data loss permanently

Training Summary

The industrial training on Cyber Security provided me with comprehensive exposure to key security concepts, tools, and practical implementation techniques used in the real world. During the training, I gained knowledge of various cyber threats, attack methodologies, network security principles, and defensive mechanisms. I learned how modern systems are protected through authentication, encryption, access control, and continuous monitoring.

The training introduced essential tools like Nmap, Burp Suite, Wireshark, and Kali Linux utilities, which helped me understand vulnerabilities, network scanning, and penetration testing techniques. I also learned about security models such as the CIA Triad, OSI layers, secure communication protocols, hashing, and cryptographic algorithms.

A major part of the training involved hands-on implementation. I developed a File Encryption–Decryption Project using Python, which included generating secure keys, encrypting files, and decrypting them using the Cryptography module. This helped me understand real-world applications of cryptography, confidentiality, secure data storage, and key management.

The sessions also covered web security fundamentals, including SQL Injection, XSS, CSRF, session hijacking, cookie theft, and server-side attacks. I learned how attackers exploit system weaknesses and how to mitigate these risks using secure coding and security best practices.

Overall, this training enhanced my practical understanding of cybersecurity fundamentals and increased my confidence in identifying, analyzing, and preventing security threats. It provided me with the skills required to pursue advanced studies, certifications, and career opportunities in the cyber security domain.

Key Outcomes

1. Understanding of Cyber Security Fundamentals
2. Hands-On Experience with Security Tools
3. Knowledge of Cyber Attacks & Their Prevention
4. Application of Cryptography in Real World
5. Improved Problem-Solving & Analytical Skills
6. Practical Experience with Python for Security
7. Preparation for Cyber Security Certifications
- 8.

Next Steps

- Learn advanced penetration testing techniques,
- Build more cybersecurity-based projects
- Gain deeper understanding of networks and protocols
- Explore advanced topics in cryptography
- Practice real-world VAPT (Vulnerability Assessment & Penetration Testing)
- Prepare for professional certifications

References

Security Tools Documentation

- Kali Linux Official Documentation – <https://www.kali.org/docs>
- Wireshark Documentation – <https://www.wireshark.org/docs>
- Nmap Reference Guide – <https://nmap.org/book/man.html>
- Burp Suite Documentation – <https://portswigger.net/burp/documentation>
- Metasploit Framework Guide – <https://docs.metasploit.com>

Python & Cryptography Resources

- Python Official Documentation – <https://docs.python.org>
- Cryptography Library Documentation – <https://cryptography.io>
- Stack Overflow (Troubleshooting & Practical Examples) – <https://stackoverflow.com>

Research Papers & Standards

- RFC 5246 – Transport Layer Security (TLS)
- RFC 4949 – Internet Security Glossary
- MITRE ATT&CK Knowledge Base – <https://attack.mitre.org>